Industrial Internship Report on

"Python URL Shortener"

Prepared by

Amit Sharma

B.Tech 3rd Year — Computer Science & Engineering (Specialization: AI & Data Science)

Indian Institute of Information Technology, Manipur (IIIT Manipur)

Executive Summary
This report provides details of the Industrial Internship provided by Upskill Campus (USC) in association with the industry partner UniConverge Technologies Pvt. Ltd. (UCT). The internship was conducted on a remote basis over a period of 6 weeks, commencing on March 6, 2026. The internship was focused on a project and problem statement provided by UCT. The entire project, including this report, was to be completed within the stipulated 6-week timeframe. My project was a Python URL Shortener — a Flask-based web application that converts long URLs into compact, shareable short links. The system uses a SQLite database to store URL mappings and handles redirection seamlessly via Python routing functions. This internship provided a highly valuable opportunity to gain exposure to real-world industrial problems and design, develop, and deploy a working software solution. The experience strengthened my technical foundation in Python, Flask, and SQLite. It was an overall enriching and professionally rewarding experience.

TABLE OF CONTENTS

1 Preface....3

2 Introduction....4

2.1 About UniConverge Technologies Pvt Ltd....4

2.2 About upskill Campus (USC)....8

2.3 The IoT Academy....9

2.4 Objectives of this Internship program....10

2.5 Reference....10

2.6 Glossary....10

3 Problem Statement....11

4 Existing and Proposed solution....12

4.1 Code submission (Github link)....12

Github link : https://github.com/amitishusharma2005-prog/upskill-Campus....12

4.2 Report submission (Github link)....12

Github link : https://github.com/amitishusharma2005-prog/upskill-Campus....12

5 Proposed Design/ Model....13

5.1 High Level Diagram....13

5.2 Low Level Diagram....14

5.3 Interfaces....15

6 Performance Test....16

6.1 Test Plan/ Test Cases....16

6.2 Test Procedure....16

6.3 Performance Outcome....16

7 My learnings....17

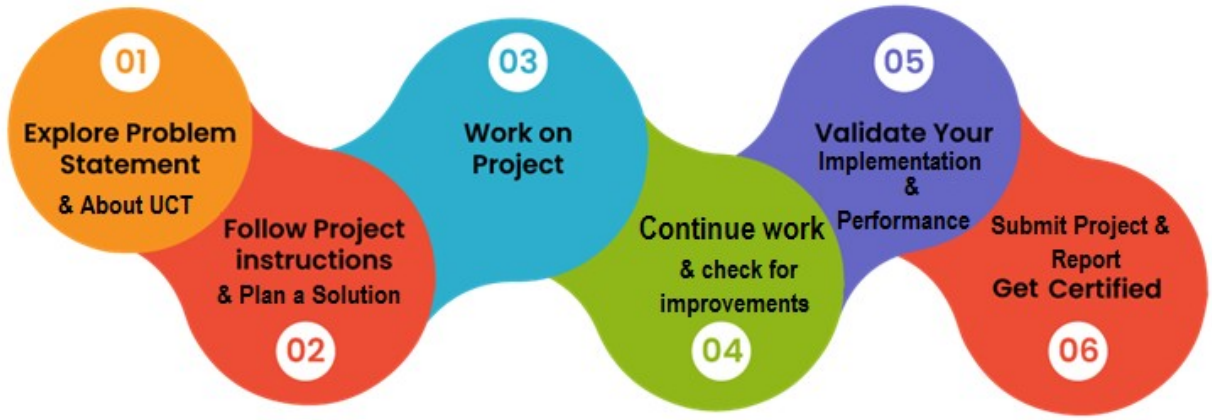
8 Future work scope....18

1 Preface

The following report documents the work carried out during my 6-week Industrial Internship as a Python Intern at UniConverge Technologies Pvt. Ltd. (UCT), facilitated by Upskill Campus (USC). This internship was a significant milestone in my academic journey as a 3rd-year B.Tech student at IIIT Manipur.

Internships serve as a critical bridge between academic learning and professional practice. Working on a real-world project enables students to apply theoretical knowledge, appreciate engineering constraints, and develop an industry-ready problem-solving mindset.

My project, the Python URL Shortener, was a full-stack web application built using Flask, SQLite, and Python. The program was planned meticulously: initial weeks focused on architecture and Flask routing, middle weeks on database integration and UI, and final weeks on testing and documentation.



I gained in-depth practical knowledge of Python web development, database operations, and software testing. I would like to express my sincere gratitude to Prabha Singh (HR Manager, UCT & USC) and the technical mentors at UCT for their invaluable guidance.

To my juniors and peers: I strongly encourage you to pursue such internships. The gap between academic study and industry practice is best bridged through direct, hands-on experience. Treat every project as an opportunity to demonstrate engineering professionalism.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.

i. UCT IoT Platform (uct Insight)

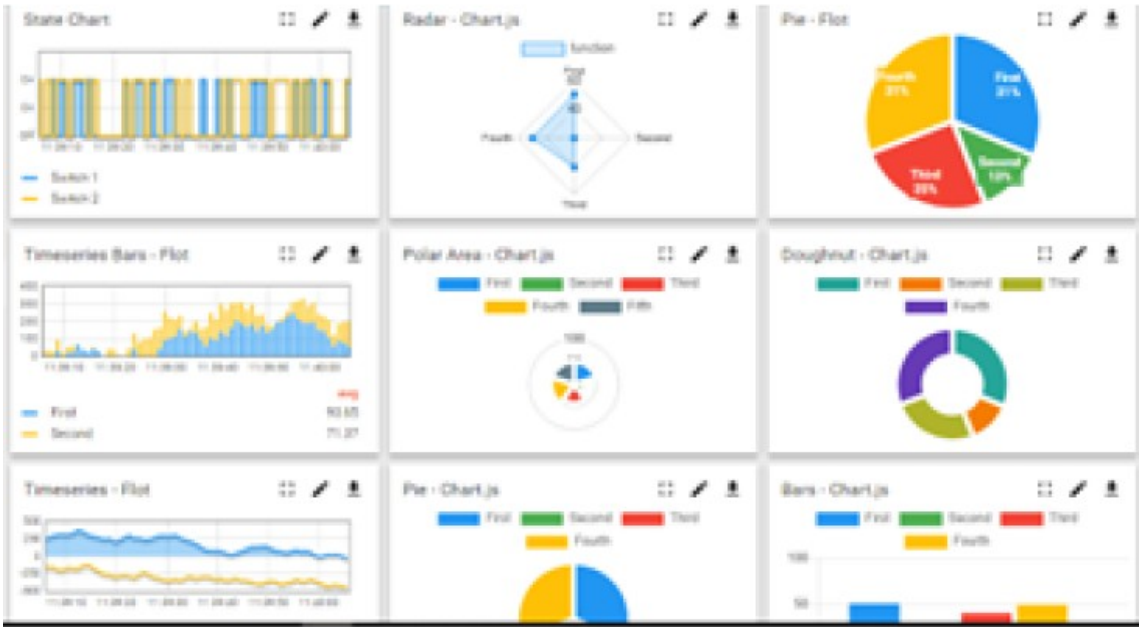
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

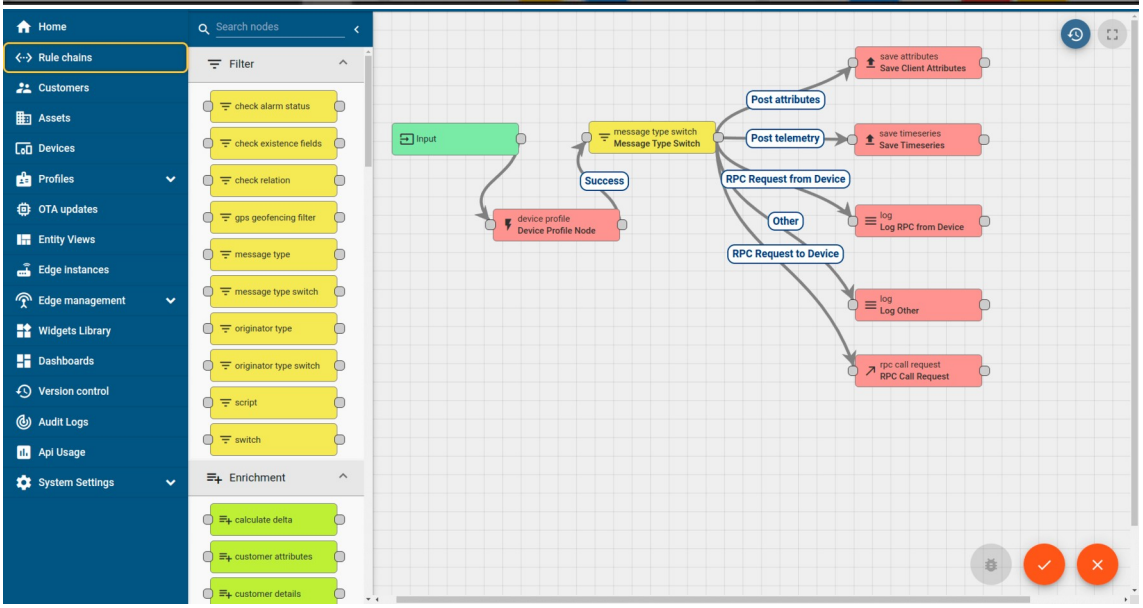
- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

• Build Your own dashboard

- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





ii. **Smart Factory Platform (**

)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.

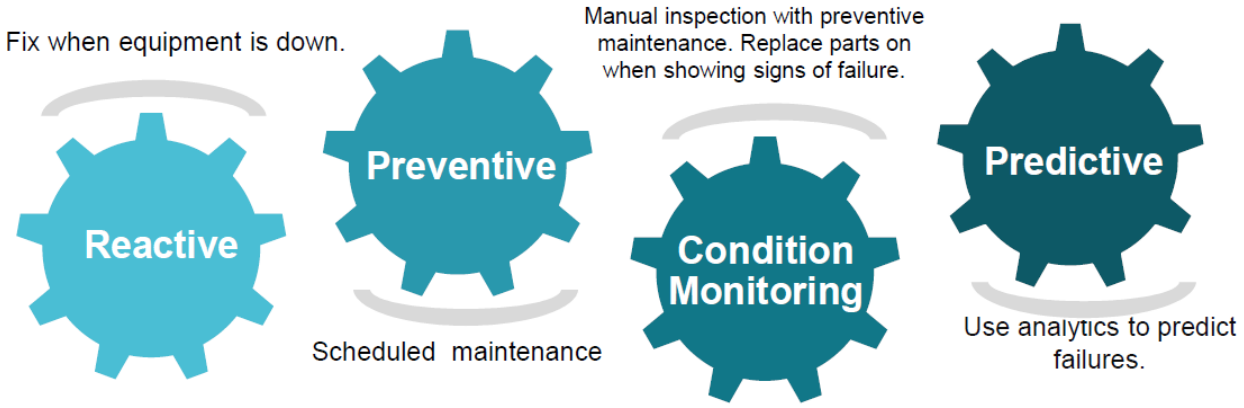
Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

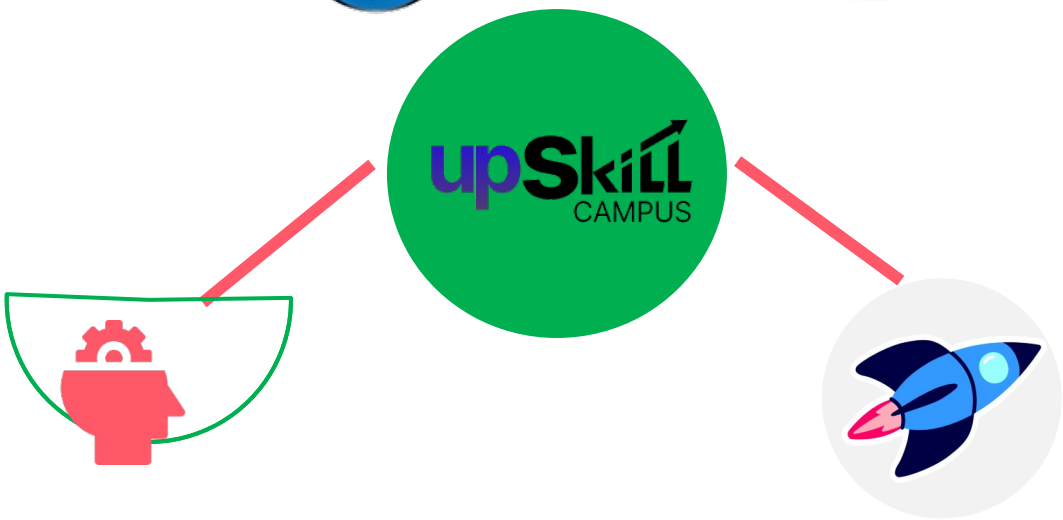
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

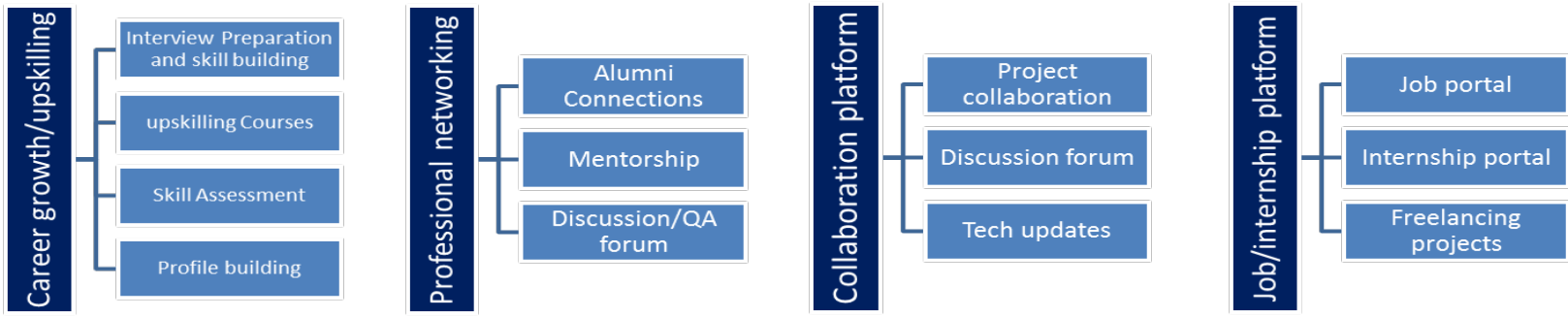
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- To gain practical experience of working in a professional industry environment.
- To solve real-world engineering and software development problems.
- To improve job prospects and career readiness through project-based learning.
- To develop an improved understanding of Python backend applications.
- To achieve personal and professional growth, including better communication, time management, and problem-solving skills.

2.5 Reference

[1] Flask Web Development Documentation — https://flask.palletsprojects.com/

[2] SQLite3 Python Standard Library Documentation — https://docs.python.org/3/library/sqlite3.html

[3] Python 3.11 Official Documentation — https://docs.python.org/

2.6 Glossary

Terms	Acronym
Flask	A lightweight Python web framework for building web applications.
SQLite	A serverless, self-contained SQL database engine used for local data storage.
URL	Uniform Resource Locator — a web address identifying a resource on the Internet.
HTTP 302	A status code indicating a temporary redirect to a different URL.
Hash	A generated alphanumeric string used to represent the original data.

3 Problem Statement

In the assigned problem statement

In the assigned problem statement, the objective was to design and implement a Python-based URL Shortener application.

Long URLs are cumbersome to share, prone to breaking in emails and messages, and difficult to memorize. The challenge was to build a self-contained, functional URL Shortener from scratch using Python.

Specifically, the scope of the project required:

1. A mechanism to accept a long URL as input from the user.

2. An algorithm to generate a unique, short alphanumeric identifier (6 characters) for each URL.

3. A persistent storage system (SQLite) to maintain the mapping between the original URL and its short identifier.

4. A redirection mechanism so that accessing the short URL automatically forwards the user to the original long URL.

5. A clean and functional web-based user interface accessible via a browser.

4 Existing and Proposed solution

Existing Solutions and Their Limitations: Several URL shortening services exist in the market, including Bitly and TinyURL. The primary limitation of all existing solutions is that they are external, cloud-hosted services. They introduce dependency on third-party platforms, create potential data privacy issues (as all links are stored on their servers), and often restrict features behind paid tiers.

Proposed Solution: The proposed solution is a self-hosted Python URL Shortener web application built using Flask as the web framework and SQLite as the lightweight database.

The value addition of this solution over existing tools includes complete data ownership (all URL mappings are stored locally in `urls.db`), zero external dependencies, and a fully customizable codebase that guarantees user data privacy.

4.1 Code submission (Github link)

Github link : https://github.com/amitishusharma2005-prog/upskill-Campus

4.2 Report submission (Github link)

Github link : https://github.com/amitishusharma2005-prog/upskill-Campus

5 Proposed Design/ Model

This section details the design flow of the Python URL Shortener solution. The system has three distinct layers: the Presentation Layer (HTML UI), the Application Layer (Flask), and the Data Layer (SQLite). Data flows from the user through the Flask application and into the database, with the reverse flow occurring on redirection.

5.1 High Level Diagram

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                    PYTHON URL SHORTENER                                      │
│                High-Level Architecture Diagram                               │
└──────────────────────────────────────────────────────────────────────────────┘
                        │
┌──────────────────┐   HTTP   ┌──────────────────────┐   SQL   ┌──────────────────┐
│  USER /          │◄────POST────►│  FLASK SERVER        │◄──────────►│  SQLite DB       │
│  BROWSER         │  (HTTP)  │  (app.py)            │ (urls.db)│ url_mapping      │
└──────────────────┘          └──────────────────────┘          └──────────────────┘
    │                    │                    │
Enter Long URL          Generate Short ID        Store Mapping
Access Short URL        Redirect to Original     Retrieve Mapping
Receive Redirect        Return 404 if not found
```

Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram

URL Shortening Data Flow:

1. **System Initialization:** `init_db()` executes ➔ `CREATE TABLE IF NOT EXISTS url_mapping`.
2. **User Request:** User accesses `GET /` ➔ Flask renders the `HTML_TEMPLATE` input form.
3. **Data Submission:** User submits a Long URL via `POST /`.
4. **Validation:** System checks for URL prefix ➔ Automatically prepends `http://` if missing.
5. **ID Generation:** `generate_short_id()` executes ➔ Randomly selects 6 characters from an alphanumeric pool [a-z, A-Z, 0-9].
6. **Database Write:** System executes ➔ `INSERT INTO url_mapping (original_url, short_url) VALUES (?, ?)`.
7. **Response:** Flask returns the generated short URL to the user's browser.

Redirection Data Flow:

1. **Link Execution:** User attempts to access the generated short link (`GET /<short_id>`).
2. **Database Lookup:** System executes ➔ `SELECT original_url FROM url_mapping WHERE short_url = ? .`
3. **Resolution (Success):** If the ID is found ➔ System issues an `HTTP 302 Redirect` to the `original_url`.
4. **Resolution (Failure):** If the ID is not found ➔ System issues an `HTTP 404 – URL not found` error.

5.3 Interfaces

User Interface: The UI is an inline HTML5 template rendered by Flask. It consists of a centered form with a text input for the long URL and a submit button. A conditional Jinja2 block displays the generated link upon success.

Database Schema: The SQLite database contains a single table initialized via: `CREATE TABLE IF NOT EXISTS url_mapping (id INTEGER PRIMARY KEY AUTOINCREMENT, original_url TEXT NOT NULL, short_url TEXT NOT NULL UNIQUE);`

Flask Routing Interface:

- `GET /`: Renders the submission form.
- `POST /`: Accepts the long URL, generates the hash, inserts it into the database, and returns the short link.
- `GET /<short_id>`: Queries the database and redirects the user to the original URL.

6 Performance Test

For the Python URL Shortener, the key constraints identified were response time, database write performance, and URL validation accuracy.

6.1 Test Plan/ Test Cases

- **TC-01:** Valid URL shortening (Input: standard https URL).
- **TC-02:** URL without scheme (Input: www.example.com without http://).
- **TC-03:** Short URL redirection (Input: Click generated short link).
- **TC-04:** Invalid short ID (Input: Random/unknown short ID).

6.2 Test Procedure

The Flask development server was launched locally. Functional tests were executed manually using a web browser to interact with the HTML interface. The SQLite3 database was inspected after each write operation to visually verify inserted records and schema integrity.

6.3 Performance Outcome

All test cases passed successfully.

- **TC-01 & TC-02:** The system successfully generated 6-character IDs and automatically prepended 'http://' to scheme-less inputs.

- **TC-03:** HTTP 302 redirect responses were consistently served with minimal latency.

- **TC-04:** Invalid IDs correctly returned a 404 Error without crashing the server. The SQLite database accurately stored all hashed pairs and efficiently handled concurrent read operations.

7 My learnings

This internship was a transformative learning experience that advanced my technical proficiency.

- **Flask Web Framework:** I gained hands-on proficiency in Flask, including route definition, handling GET and POST methods, rendering templates, and understanding the HTTP request-response lifecycle.

- **SQLite Database Management:** I learned to manage a SQLite database programmatically, including table creation, parameterized `INSERT` queries for security, and `SELECT` queries.

- **Professional Discipline:** Working within a fixed timeline taught me to prioritize features, allocate time effectively, and document my code professionally. These skills directly complement my AI & Data Science coursework at IIIT Manipur.

8 Future work scope

While the current application is a fully functional Minimum Viable Product, several enhancements can be pursued in the future:

- **Custom Short URLs:** Allow users to specify their own desired short ID (e.g., `domain.com/my-link`) instead of relying solely on random generation.

- **Click Analytics:** Track the number of times each short URL is accessed by adding a visits column to the database to display usage statistics.

- **User Authentication:** Add user account management so that users can manage, view, or delete their own shortened URLs.

- **Database Migration:** Migrate the backend from SQLite to PostgreSQL to handle high-traffic production deployments and concurrent writes.